

LAPORAN PRAKTIKUM ALGORITMA PEMROGRAMAN
PEKAN 6: WHILE LOOP, DO WHILE LOOP, SENTINEL LOOP, METHOD



OLEH:
ALFARIS AULIA RAHMAN
2411533006

DOSEN PENGAMPU:
DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI
DEPARTEMEN INFORMATIKA
UNIVERSITAS ANDALAS
2024

a. Pendahuluan

1. While Loop

While loop adalah salah satu struktur kontrol perulangan yang digunakan untuk menjalankan blok kode secara berulang selama kondisi tertentu bernilai benar. Kondisi ini diperiksa sebelum perulangan dimulai, sehingga jika kondisi tidak terpenuhi sejak awal, perulangan tidak akan dijalankan sama sekali. Struktur ini sering digunakan ketika jumlah iterasi tidak diketahui dan hanya bergantung pada perubahan kondisi tertentu di dalam atau di luar perulangan.

2. Do While Loop

Do-while loop adalah variasi dari while loop yang memastikan blok kode selalu dijalankan setidaknya satu kali, terlepas dari apakah kondisi awal bernilai benar atau salah. Hal ini terjadi karena pengecekan kondisi dilakukan setelah eksekusi blok kode pertama. Do-while loop sering digunakan ketika diperlukan jaminan bahwa kode tertentu harus dieksekusi setidaknya satu kali sebelum memutuskan untuk melanjutkan atau menghentikan perulangan.

3. Sentinel Loop

Sentinel loop adalah jenis perulangan yang berhenti ketika menemukan nilai khusus yang disebut sentinel. Sentinel ini digunakan sebagai penanda untuk mengakhiri proses, sehingga cocok untuk situasi di mana jumlah iterasi tidak diketahui sebelumnya. Contohnya adalah membaca data dari pengguna hingga nilai sentinel, seperti angka nol atau karakter tertentu, dimasukkan untuk menghentikan proses.

4. Method

Method dalam Java adalah blok kode yang dirancang untuk menyelesaikan tugas tertentu, memungkinkan program lebih terstruktur, mudah dibaca, dan dikelola. Method terdiri dari deklarasi yang mencakup *access modifier*, tipe pengembalian, nama, dan parameter (jika ada). Setelah itu, terdapat badan method yang berisi pernyataan-pernyataan untuk menjalankan logika tertentu. Method dapat mengembalikan nilai melalui *return statement* atau tidak mengembalikan apa pun jika bertipe `void`. Dengan menggunakan method, kita dapat menghindari pengulangan kode, mendukung penggunaan ulang logika yang sama di berbagai bagian program, dan memisahkan program menjadi bagian-bagian kecil yang lebih modular.

b. Tujuan

Tujuan dari praktikum ini adalah untuk mempelajari dan menerapkan dasar-dasar while loop, sentinel loop, dan do-while loop dalam bahasa pemrograman Java.

c. Langkah Pengerjaan Praktikum

a. While Loop

1. Buat *package* dan *class* baru, lalu beri nama *package* sebagai “pekan6”, dan untuk nama class dengan nama “PerulanganWhile1”
2. Impor Scanner untuk meng-input nilai dengan memasukkan “import java.util.Scanner”.

```
package pekan6;
```

```
import java.util.Scanner;
```

```
public class PerulanganWhile1 {
```

3. Inisialisasikan variabel yang digunakan, yaitu “counter” dengan data type int, variabel “jawab” dengan data type String, variabel “running” dengan data type boolean. Inisialisasikan Scanner dengan memberikan variabel “scan”.

```
public static void main(String[] args) {
```

```
    int counter=0;
```

```
    String jawab;
```

```
    boolean running = true;
```

```
    //deklarasi scanner
```

```
    Scanner scan = new Scanner(System.in);
```

4. Tambahkan while loop dengan menambahkan “while”, yang diikuti dengan kondisi yang diperlukan di dalam tanda kurung. Tuliskan perintah untuk menghitung iterasi dan menampilkan pesan di Console.

```
    while (running) {
```

```
        counter++;
```

```
        System.out.println("Jumlah = "+counter);
```

```
        System.out.println("Apakah lanjut (ya / tidak?)");
```

```
        jawab= scan.nextLine();
```

```
        //cek jawab = tidak, perulangan berhenti
```

```
        if (jawab.equalsIgnoreCase("tidak")) {
```

```
            running=false;
```

```
        }
```

```
}
```

5. Tambahkan *Scanner* untuk menerima input dari user dan memeriksa apakah perulangan harus berhenti.
6. Setelah perulangan selesai, tampilkan jumlah iterasi.

```
System.out.println("Anda sudah melakukan perulangan  
sebanyak "+counter+" kali");  
}
```

7. Klik Run untuk menjalankan program.

Hasil output:

Jumlah = 1

Apakah lanjut (ya / tidak?)

ya

Jumlah = 2

Apakah lanjut (ya / tidak?)

tidak

Anda sudah melakukan perulangan sebanyak 2 kali

b. Do While Loop

1. Buat *class* baru, beri nama “doWhile1”.
2. Impor Scanner dengan memasukkan “import java.util.Scanner”.

```
package pekan6;  
import java.util.Scanner;  
public class doWhile1 {
```

3. Inisialisasikan variabel *phrase* dengan data type string, kemudian inisialisasi Scanner dengan nama “console”.

```
public static void main(String[] args) {  
    Scanner console = new Scanner (System.in);  
    String phrase;
```

4. Gunakan *do-while loop* untuk meminta pengguna memasukkan password. Variabel bernama "phrase" digunakan untuk menyimpan input string dari pengguna. Program akan terus meminta input hingga pengguna memasukkan password yang benar, yaitu "abcd".

```
do {
```

```

        System.out.print("Input Password: ");
        phrase = console.next();
    } while (!phrase.equals("abcd"));

```

```

    }

```

5. Klik run untuk menjalankan program

Hasil output:

Input Password: a

Input Password: ab

Input Password: abc

Input Password: abcd

c. Sentinel Loop

1. Buat *class* baru. Beri nama “SentinelLoop”
2. Impor Scanner dengan memasukkan “import java.util.Scanner”.

```

package pekan6;

import java.util.Scanner;

public class SentinelLoop {

```

3. Inisialisasikan variabel yang dibutuhkan, yaitu “sum” dan “num”, dengan tipe data int. Atur nilai awal *sum* menjadi 0, sedangkan “num” diatur ke angka berapa saja selain 0.

```

    public static void main(String[] args) {
        Scanner console = new Scanner (System.in);

        int sum = 0;

        int number = 12; //"dummy value", anything but 0

```

4. Tulis struktur *while loop* untuk memproses input selama nilai penanda berhenti (sentinel) yang berupa 0 belum dimasukkan.

```

    while (number !=0) {
        System.out.print("Masukkan angka (0 untuk keluar):
        ");

        number = console.nextInt();

        sum = sum + number;

```

5. Setelah seluruh perulangan selesai, tampilkan jumlah keseluruhan dari semua angka yang telah dimasukkan.

```

        System.out.println("Totalnya adalah " + sum);
    }

```

6. Klik Run untuk menjalankan program.

Hasil output:

Masukkan angka (0 untuk keluar): 1

Masukkan angka (0 untuk keluar): 2

Masukkan angka (0 untuk keluar): 0

Totalnya adalah 3

- d. Permainan Lempar Dadu

1. Buat *class* baru. Beri nama “Lempardadu”.
2. Import *Random* dari *java.util* untuk membuat angka acak.

```

package pekan6;

import java.util.Random;

public class Lempardadu {

```

3. Tentukan variabel yang dibutuhkan: "int tries" untuk mencatat jumlah percobaan, "int sum" untuk menyimpan total penjumlahan dari kedua dadu, dan "Random rand" sebagai generator angka acak.

```

        Random rand = new Random();

        int tries = 0;

        int sum = 0;

```

4. Buatlah perulangan *while* yang terus berjalan hingga nilai *sum* mencapai 7.
5. Selanjutnya, di dalam perulangan *while*, hasilkan dua angka acak antara 1 dan 6 untuk mewakili dua dadu. Tambahkan kedua angka tersebut dan simpan ke dalam variabel “sum”. Cetak hasil lemparan dadu ke layar. Tingkatkan nilai “tries” setiap kali perulangan dijalankan.

```

while (sum != 7) {
    //roll the dice once

    int dadu1= rand.nextInt(6) + 1;
    int dadu2= rand.nextInt(6) + 1;
    sum = dadu1 + dadu2;
    System.out.println(dadu1 + " + " + dadu2 + " = " +
        sum);
    tries++;
}

```

```
}
```

6. Setelah keluar dari perulangan, cetak jumlah percobaan yang dibutuhkan untuk mencapai angka 7.

```
System.out.println("Kamu menang setelah " + tries + "  
percobaan!");
```

```
}
```

7. Klik Run untuk menjalankan program.

Hasil output:

2 + 5 = 7

Kamu menang setelah 1 percobaan!

e. Method

1. Buat class baru. Beri nama “GamePenjumlahan”.
2. Import *Scanner* dari java.util.Scanner dan Random dari java.util.Random package pekan6;
import java.util.Random;
import java.util.Scanner;
public class GamePenjumlahan {
3. Inisialisasikan *Scanner* dengan nama “console”. Kemudian inisialisasikan Random dengan nama “rand”
4. Selanjutnya, deklarasikan variabel "points" dan "wrong" dengan tipe data int dan tetapkan nilainya menjadi 0.

```
public static void main(String[] args) {  
    Scanner console = new Scanner (System.in);  
    Random rand = new Random();  
    //play until user gets 3 wrong  
    int points = 0;  
    int wrong = 0;
```

5. Buatlah perulangan while dengan kondisi *true* jika variabel "wrong" kurang dari 3. Selanjutnya, deklarasikan variabel "result" dengan tipe data int dan tetapkan nilainya menggunakan *public class* "play".
6. Lalu, gunakan *if statement* dengan kondisi *True* jika “result” lebih besar dari 0. Di dalam *if statement*, tambahkan variabel “points” jika kondisi tersebut

terpenuhi, dan tambahkan variabel “wrong” jika kondisi tidak terpenuhi.

Setelah keluar dari perulangan, tampilkan jumlah poin yang diperoleh.

```
while (wrong < 3) {  
    int result = play(console, rand); //play one game  
    if (result > 0) {  
        points++;  
    } else {  
        wrong++;  
    }  
}
```

7. Kemudian, deklarasikan metode public static int di luar metode public static void dengan nama "play" yang memiliki parameter Scanner console dan Random rand.
8. Selanjutnya, di dalam *public class* “play”, deklarasikan variabel “operands” dengan tipe data int dan tetapkan nilainya dengan `rand.nextInt(4) + 2`. Setelah itu, deklarasikan variabel “sum” dengan tipe data yang sama, kemudian tetapkan nilainya dengan `rand.nextInt(10) + 1``.

```
public static int play(Scanner console, Random rand) {  
    int operands = rand.nextInt(4) + 2;  
    int sum = rand.nextInt(10) + 1;
```

9. Kemudian, tampilkan nilai "sum".
`System.out.print(sum);`
10. Tuliskan perulangan for yang dimulai dari 2 dengan kondisi kurang dari atau sama dengan “operands”. Di dalam perulangan for, deklarasikan variabel “n” dengan tipe data int dan tetapkan nilainya dengan `rand.nextInt(10) + 1`. Kemudian, tambahkan variabel “n” ke dalam variabel “sum”. Tampilkan nilai variabel “n” dengan tanda “+”.

```
for (int i = 2; i <= operands; i++) {  
    int n = rand.nextInt(10) + 1;  
    sum += n;  
    System.out.print(" + " + n);  
}
```

11. Setelah keluar dari for loop, tampilkan tanda "=". Kemudian, deklarasikan variabel "guess" dengan tipe data int dan gunakan Scanner untuk menerima input integer dari pengguna.
12. Selanjutnya, gunakan *if/else statement* untuk memeriksa apakah jawaban pengguna sesuai dengan nilai variabel "sum". Jika sesuai, *return* nilai 1; jika tidak, *return* nilai 2.

```
System.out.print(" = ");  
  
//read user's guess and report whether it was correct  
    int guess = console.nextInt();  
    if (guess == sum) {  
        return 1;  
    } else {  
        System.out.println("Wrong! The answer was " +  
            sum);  
        return 0;  
    }  
}
```

13. Klik Run untuk menjalankan program.

Hasil output:

7 + 9 + 1 + 10 + 10 = 37

3 + 6 + 9 + 6 + 3 = 14

Wrong! The answer was 27

6 + 2 + 6 + 5 + 10 = 12

Wrong! The answer was 29

9 + 2 + 8 + 6 = 12

Wrong! The answer was 25

You earned 1 total points.

d. Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa dalam bahasa pemrograman Java, while loop, do-while loop, dan sentinel loop memiliki peran penting dalam mengelola proses perulangan dengan fleksibilitas tertentu. While loop menjalankan perulangan selama kondisi tetap bernilai benar, do-while loop memastikan blok kode dieksekusi setidaknya satu kali sebelum memeriksa kondisi, dan sentinel loop berhenti saat mendeteksi nilai khusus sebagai penanda (sentinel). Selain itu, penerapan method memungkinkan program dibagi menjadi bagian-bagian kecil yang lebih terorganisasi, mendukung penggunaan ulang kode, dan meningkatkan keterbacaan program.